

Predicting Kickstarter Project Success from Launch-Time Information

UE24CS352A Machine Learning Mini-Project | Team #18 | Problem #56 | Cheethirala Sai Santosh (PES1UG24CS127), Chhavi Siddarth Wadhwa (PES1UG24CS132)

1. Problem statement

Given only information available *when a Kickstarter campaign launches* (funding goal, category, country, currency, creator location, campaign length, launch date, and the project name and short blurb), predict whether the campaign will reach its funding goal. This is binary classification. We follow Khosla, Reinecke and Wittenbrink (Stanford, *Crowdfunding: Predicting Kickstarter Project Success*), who report about 83% accuracy, and compare against their results.

2. Dataset

We use the public Web Robots Kickstarter scrape (monthly cumulative snapshots). Reading 2,144,137 rows gave 231,080 unique project ids (latest snapshot kept per id). We keep finished campaigns only (successful or failed; live and canceled removed), drop rows with missing goal or dates, and de-duplicate on (name, blurb, launch date, deadline) as in the paper, leaving **47,765 projects** launched 2024-07-01 to 2026-06-04 (70.9% succeeded; majority-class accuracy 70.7%). Goals are converted to USD (median \$3,000). The split is a random 70/30 split: 33,435 train / 14,330 test. Post-launch fields (backers, pledged amount, staff pick) are never used, since they would leak the outcome.

3. Approach

Features. (a) Metadata: log goal, campaign length, launch month and weekday, name/blurb lengths, one-hot parent category and category; (b) **out-of-fold target encodings** (5 folds, smoothed) of currency, country, category, parent category, location and location type, plus the mean goal per level and the project's goal relative to it; (c) text: TF-IDF unigrams of the blurb fed to Multinomial Naive Bayes whose **out-of-fold** success probability becomes a feature, and optional VADER sentiment, 20-topic LDA, and 50-dimensional LSA embedding.

Models. Gradient boosting (LightGBM), random forest, one-hidden-layer neural net, linear SVM, logistic regression, and ridge / OLS / lasso linear probability models thresholded at 0.5. All use balanced sample weights where supported. **Protocol.** Fit on train only; report accuracy, F1, per-class precision/recall and ROC-AUC on the untouched test set; text-feature ablation over 5 seeds. **Differences from the paper:** VADER instead of Stanford CoreNLP, LSA instead of pretrained Word2Vec, and scikit-learn's ReLU MLP instead of a SELU network.

4. Implementation overview

Python (pandas, scikit-learn, LightGBM). `src/data.py` loads and cleans the dump; `src/features.py` builds leakage-safe features (a `FeatureBuilder` that is fitted on train and reused unchanged for test and the demo); `src/models.py` defines the model zoo and metrics; `src/train.py` runs all experiments and writes tables, figures and the saved model; `app.py` is a Streamlit demo (probability, goal what-if curve, per-prediction feature contributions); `tests/` checks leakage guards and the pipeline end to end.

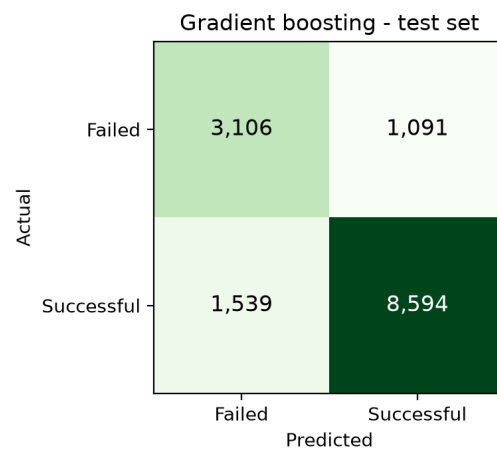
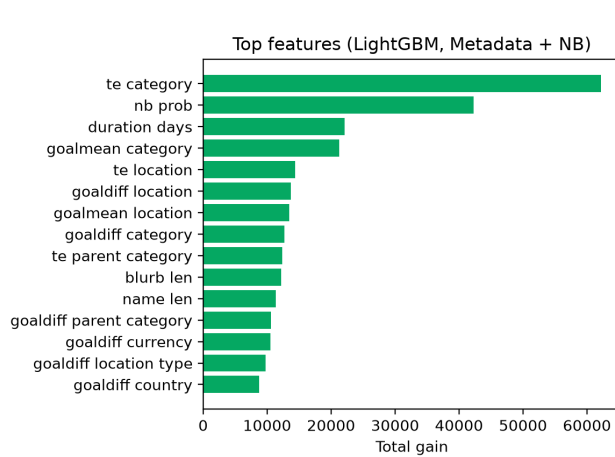
5. Results (test set)

Table 1. Models on Metadata + NB features (Sample 2). Class 1 = successful.

Model	Acc	F1	P1	P0	R1	R0	AUC
G. Boosting	0.816	0.867	0.887	0.669	0.848	0.740	0.887
Logistic	0.797	0.847	0.906	0.618	0.794	0.802	0.879
R. Forest	0.796	0.845	0.910	0.615	0.789	0.812	0.885
SVM	0.796	0.846	0.906	0.616	0.793	0.802	0.878
Neural Net	0.794	0.845	0.906	0.614	0.791	0.801	0.882
Ridge	0.792	0.843	0.905	0.611	0.789	0.800	0.877
OLS	0.792	0.843	0.905	0.611	0.789	0.800	0.877
Lasso	0.792	0.843	0.906	0.610	0.788	0.802	0.877

Text-feature ablation (LightGBM accuracy, mean over seeds): Metadata 80.93%; Metadata + NB 81.77%; Metadata + NB + Sentiment 81.74%; Metadata + NB + LDA 82.30%; Metadata + NB + LSA 82.33%.

Significance (paired bootstrap on the same 14,330 test rows, one model seed, so estimates differ slightly from the 5-seed means above; accuracy change in points with 95% CI): NB +0.68 [0.22, 1.14]; Sentiment +0.10 [-0.29, 0.45] (not significant); LDA +0.79 [0.37, 1.19]; LSA +0.68 [0.17, 1.14]. Gradient boosting leads each of the other 7 models by 2.0 to 2.5 points (McNemar $p < 1e-13$).



6. Conclusions

- **G. Boosting** is the best model: 81.6% accuracy, F1 0.867, AUC 0.887, +10.9 points over the majority-class baseline (paper: about 83% accuracy).
- The out-of-fold Naive-Bayes blurb probability lifts accuracy from 80.9% to 81.8% (+0.84 points). Measured against metadata + NB, VADER sentiment moves it -0.03 points, LDA +0.53 and LSA +0.56. A one-line blurb supports one well-regularised text feature; the largest change from any further text block is 0.56 points, so the extras refine that feature rather than drive the result.
- The best model leads the fourth-best by 2.1 points, so model choice matters here.
- For gradient boosting (confusion matrix above), recall on failed projects is 0.74 vs 0.85 on successful ones: failures are harder to catch than successes. 1,539 successes were predicted as failures and 1,091 failures as successes.
- **Limitations / future work:** launch-time metadata and a one-line blurb are limited signals; the long project story, images and creator history are not used. A temporal split (`--split temporal`) gives a more realistic estimate under platform drift. Transformer sentence embeddings are the natural next step.